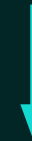


Missing Values, Duplicates, and Data Types

Python 101 • Day 6 • Data Quality with pandas

Today's move

from “the file
loaded”



to “the data is
trustworthy”

missing • duplicate •
typed • valid

BLOCK 43

What this block covers

30 minutes instruction + 30 minutes guided practice

Missing values

Find blanks, NaN values, and failed conversions.

Duplicates

Separate exact duplicate rows from repeated identifiers.

Data types

Convert text columns into numeric values carefully.

Quality report

Describe what was discovered before cleaning.

Guiding question: What evidence tells us this table can be trusted?

Real data is often inconsistent

A CSV can load successfully while still containing serious problems.

record_id	source	status	count	complete	errors
EQ-1001	maint	active	12000	100	0
EQ-1002	log	active	8500	95	2
EQ-1003	finance		5000	100	0
EQ-1004	maint	active	unknown	80	3
EQ-1004	maint	active	unknown	80	3

Problems

Missing status

Invalid numeric text

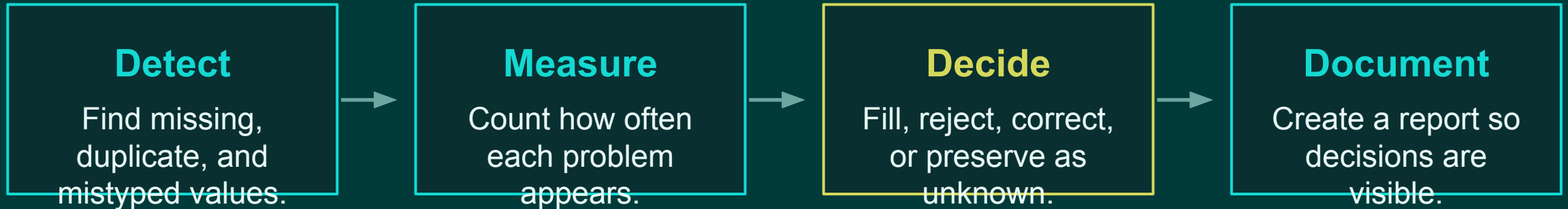
Exact duplicate row

Repeated record ID

Loading the file only proves Python could read the file. It does not prove the data is correct.

Data quality is an investigation

Before cleaning, discover and count the problems.



Do not start by deleting rows. Start by understanding what is wrong.

Missing values

Pandas often displays missing values as NaN.

Practical question

Do not worry about the internal details yet.
For now ask:

Is this value missing?

Examples students may see

NaN

None

blank cell in a CSV

failed numeric conversion

BLOCK 43

Finding missing values with `isna()`

A missing-value check creates a Boolean Series.

```
missing_status = (  
    records["status"].isna()  
)  
  
print(missing_status.head())
```

Result is a mask

row	status	isna
0	active	False
1	active	False
2	NaN	True
3	active	False

True marks rows where the value is missing.

BLOCK 43

Counting missing values in one column

A Boolean mask can be summed because True behaves like 1.

```
missing_status_count = (  
    records["status"]  
    .isna()  
    .sum()  
)  
print(missing_status_count)
```

Why .sum() works

mask	number
True	1
False	0

This is the same Boolean idea students learned earlier, applied to a whole column.

BLOCK 43

Missing values across the table

This is the beginning of a data-quality report.

```
missing_by_column = (  
    records.isna().sum()  
)  
  
print(missing_by_column)
```

record_id	1
source	0
status	4
record_count	2
completion	0
error_count	0

Now we know where to investigate.

A count is not a cleaning decision. It is evidence.

Missing does not automatically mean delete

The right choice depends on what the field means.

Required field

Missing record_id

The row may be impossible to trace, join, or audit.

Likely decision: reject or isolate.

Optional field

Missing description

The value may be unknown but not fatal.

Likely decision: keep, mark unknown, or fill from a trusted source.

Ask: would filling the value invent information?

Filling missing values

Use fillna only when the replacement honestly represents the data.

```
records["status"] = (  
    records["status"]  
    .fillna("unknown")  
)
```

When this is appropriate

“unknown” must mean:

We genuinely do not know the status.

It must not mean:

The programmer wanted errors to disappear.

Filling is a data change. Treat it as a decision, not a shortcut.

Dropping versus isolating bad rows

For required fields, preserving rejected rows is often safer.

Remove rows

```
clean_records = records.dropna(  
    subset=["record_id"]  
)
```

Simple, but it can hide what was removed.

Isolate rows

```
missing_id = records[  
    records["record_id"].isna()  
]  
  
usable_records = records[  
    records["record_id"].notna()  
]
```

Better for audits and summaries.

BLOCK 43

Duplicate rows

An exact duplicate row repeats every value in the row.

```
duplicate_mask = records.duplicated()

duplicate_count = duplicate_mask.sum()

duplicates = records[
    duplicate_mask
]
```

record_id	status	errors
EQ-1004	active	3
EQ-1004	active	3

Every visible field matches.

drop_duplicates() can remove exact duplicates, but decide before changing the data.

Duplicate business keys

A repeated identifier can be more serious than an exact duplicate row.

```
duplicate_ids = records[
    records.duplicated(
        subset=["record_id"],
        keep=False
    )
]
```

Same record_id, different facts

record_id	count	errors
EQ-1004	9000	3
EQ-1004	12000	0

Which one is true? That requires a rule or investigation.

This prepares us for primary keys and joins later today.

Incorrect data types

CSV values often arrive as text, even when they look numeric.

Column values

12000
8500
unknown
5000

The word “unknown” prevents the column from being cleanly numeric.

Important question

Can this value be represented as a number?

That is conversion, not full validation.

Converting numeric data

`pd.to_numeric()` turns numeric text into actual numeric values.

```
records["record_count"] = (  
    pd.to_numeric(  
        records["record_count"],  
        errors="coerce"  
    )  
)
```

errors="coerce"

before	after
"12000"	12000
"8500"	8500
"unknown"	NaN

Failed conversions become missing values.

Finding failed conversions

After conversion, missing values reveal invalid numeric text.

```
invalid_counts = records[
    records["record_count"].isna()
]
print(invalid_counts)
```

Conceptual flow

raw text → convert → NaN

NaN after conversion means: this value could not become a number.

Conversion is not validation

A number can convert successfully and still be invalid for the field.

Conversion question

Can it become a number?

`"-500"` → `-500`

Validation question

Is it acceptable here?

`record_count <= 0` → `invalid`

Separate “can parse” from “should accept.”

Range validation with masks

Create explicit masks for each business rule.

```
invalid_record_count = records[
    records["record_count"] <= 0
]

invalid_completion = records[
    (records["completion"] < 0)
    | (records["completion"] > 100)
]

invalid_error_count = records[
    records["error_count"] < 0
]
```

Why separate masks?

Each mask tells one story.

That makes the data-quality report
clearer and the code easier to
debug.

Avoid one giant mystery filter.

A useful cleaning sequence

The exact order can vary, but the work should be intentional.



Clean in stages so each decision can be checked before the next stage depends on it.

Quality report versus cleaned data

The report describes what was discovered; the cleaned dataframe is the result.

Data-quality report

Rows received: 1000
Missing record IDs: 3
Exact duplicate rows: 8
Duplicate record IDs: 12
Invalid record counts: 5

Evidence and counts

Cleaned dataframe

Rows retained
Values corrected or filled
Rejected rows separated
Types converted
Duplicates handled

Processed output

BLOCK 43

Guided exercise setup

Start with an intentionally inconsistent dataframe.

```
import pandas as pd

records = pd.read_csv(
    "data/raw/equipment_dirty.csv"
)

print(records.head())
print(records.shape)
print(records.dtypes)
```

Exercise goal

Do not clean yet.

First discover missing values, duplicates, type problems, and invalid ranges.

Total exercise block: 30 minutes

BLOCK 43

Exercise 1: count missing values

Goal: identify columns with the most missing values.

```
missing_counts = records.isna().sum()

print(missing_counts)

# Challenge:
# Which three columns have
# the most missing values?
```

Record your findings

Column name
Count missing
Is the field required?

Timebox: 5 minutes

BLOCK 43

Exercise 2: isolate missing required IDs

Goal: find the problem rows without deleting them.

```
missing_id = records[
    records["record_id"].isna()
]

usable_id = records[
    records["record_id"].notna()
]

print(missing_id)
print(usable_id.shape)
```

Answer

How many rows would be rejected for missing record_id?

Why should we keep them visible?

Timebox: 5 minutes

BLOCK 43

Exercise 3: find exact duplicate rows

Goal: count rows where every column repeats.

```
duplicate_mask = records.duplicated()
duplicate_count = duplicate_mask.sum()
duplicate_rows = records[
    duplicate_mask
]
print(duplicate_count)
print(duplicate_rows)
```

Question

Are these duplicated because the file repeated a row, or because the real world has two similar items?

Timebox: 4 minutes

Exercise 4: find duplicate record IDs

Goal: detect repeated business identifiers, not just identical rows.

```
duplicate_ids = records[
    records.duplicated(
        subset=["record_id"],
        keep=False
    )
]
print(duplicate_ids.sort_values("record_id"))
```

Compare

How many duplicate IDs are also exact duplicate rows?

How many duplicate IDs disagree with each other?

Timebox: 5 minutes

BLOCK 43

Exercise 5: convert numeric columns

Goal: reveal invalid numeric text by coercing failed conversions to NaN.

```
numeric_columns = [  
    "record_count",  
    "completion",  
    "error_count",  
]  
  
for column in numeric_columns:  
    records[column] = pd.to_numeric(  
        records[column],  
        errors="coerce"  
    )
```

After conversion

```
failed = records[  
    records[column].isna()  
]
```

Count failed conversions for each
numeric column.

Timebox: 5 minutes

BLOCK 43

Exercise 6: validate numeric ranges

Goal: create separate masks for each range rule.

```
bad_count = records[
    records["record_count"] <= 0
]

bad_completion = records[
    (records["completion"] < 0)
    | (records["completion"] > 100)
]

bad_errors = records[
    records["error_count"] < 0
]
```

Create counts

Invalid record counts
Invalid completions
Invalid error counts

Then inspect each dataframe.

Timebox: 6 minutes

Applied challenge: build a data-quality summary

Summarize what was discovered before cleaning the dataframe.

Required counts

- Rows received
- Missing record IDs
- Exact duplicate rows
- Duplicate record IDs
- Invalid record counts
- Invalid completion values
- Invalid error counts

Suggested output

```
quality_summary = {  
    "rows_received": len(records),  
    "missing_ids": len(missing_id),  
    "exact_duplicates": duplicate_count,  
    "invalid_counts": len(bad_count),  
}
```

Print the summary or save it for the next block.

Success: the report explains the quality of the input data.

Block 43 wrap-up

You can now inspect data quality before cleaning data.

Missing

Use `isna()`, `notna()`, and `sum()` to locate gaps.

Duplicates

Check exact rows and repeated business identifiers.

Types

Use `pd.to_numeric()` and validate ranges afterward.

A good data pipeline does not just produce clean output. It explains what was wrong with the input.

Next block: string cleaning, dates, allowed values, and building the clean dataframe.